

Arquivo: sharding.py

```
# """
# DESCRIÇÃO GERAL:
# Este módulo decide em qual shard um arquivo deve ser colocado.
# No Marco 2, a estratégia mais simples é usar hashing do caminho lógico do arquivo
# e mapear o resultado para um nó do cluster.
# """

# from hashlib import sha256

# from dfs.cluster.node_registry import NodeRegistry, NodeInfo

# class ShardManager:
#     """
#     Faz o mapeamento entre caminho lógico e nó responsável.
#     """

#     def __init__(self, registry: NodeRegistry | None = None):
#         # Reaproveita o cadastro dos nós.
#         self.registry = registry or NodeRegistry()

#     def shard_id_for_path(self, path: str) -> int:
#         """
#         Calcula o shard responsável por um caminho.
#         """
#         # Gera um hash estável do caminho.
#         digest = sha256(path.encode("utf-8")).digest()

#         # Usa os primeiros bytes do hash como inteiro.
#         value = int.from_bytes(digest[:8], byteorder="big", signed=False)

#         # Aplica a divisão modular para escolher o shard.
#         return value % self.registry.size()

#     def node_for_path(self, path: str) -> NodeInfo:
#         """
#         Retorna o nó responsável pelo caminho informado.
#         """
#         shard_id = self.shard_id_for_path(path)
#         return self.registry.get_by_index(shard_id)

#     def shard_id_for_node(self, node_id: str) -> int:
#         """
#         Retorna o índice de shard associado ao nó.
#         """
#         return self.registry.index_of(node_id)

#     def node_for_shard(self, shard_id: int) -> NodeInfo:
#         """
#         Retorna o nó correspondente a um shard.
#         """
```

```
#         return self.registry.get_by_index(shard_id)

from hashlib import sha256
from dfs.cluster.node_registry import NodeRegistry, NodeInfo

class ShardingManager:
    """
    Implementa sharding físico por chunks

    Aqui o arquivo é dividido em blocos físicos
    Cada chunk pode ir para um nó diferente

    Isso prepara o DFS para:
    - paralelismo
    - arquivos grandes
    - balanceamento melhor
    - replicação futura por chunk
    """

    def __init__(self, registry: NodeRegistry | None = None):
        # Reaproveita o cadastro dos nós
        self.registry = registry or NodeRegistry()

    # Gera um hash estável do caminho lógico para garantir que o mesmo arquivo sempre tenha a mesma distribuição
    def _stable_hash(self, text: str) -> int:
        digest = sha256(text.encode("utf-8")).digest()
        # Usa os primeiros bytes do hash como um inteiro para obter um valor numérico consistente para o cálculo
        return int.from_bytes(digest[:8], byteorder="big", signed=False)

    # Calcula o shard base para um caminho lógico, que é usado como ponto de partida para distribuir os chunks
    def base_shard_for_path(self, path: str) -> int:
        # Aplica a divisão modular para escolher o shard base, garantindo que o mesmo caminho sempre gere o mesmo shard base
        return self._stable_hash(path) % self.registry.size()

    def shard_for_chunk(self, path: str, chunk_id: int) -> int:
        """
        Distribui chunks de forma determinística

        A estratégia usada é:
            base_shard = hash(path) % total_nodes
            shard_id = (base_shard + chunk_id) % total_nodes

        Assim:
        - o mesmo arquivo sempre gera a mesma distribuição
        - chunks consecutivos tendem a cair em nós diferentes
        - o balanceamento inicial é simples
        """
        base = self.base_shard_for_path(path)
        return (base + chunk_id) % self.registry.size()

    # Retorna o nó responsável por um chunk específico de um arquivo
```

```
def node_for_chunk(self, path: str, chunk_id: int) -> NodeInfo:
    # Calcula o shard para o chunk usando a função de distribuição e retorna o nó correspondente
    shard_id = self.shard_for_chunk(path, chunk_id)
    return self.registry.get_by_index(shard_id)

def chunk_storage_path(self, path: str, chunk_id: int) -> str:
    """
    Gera o caminho físico do chunk dentro do storage node
    O caminho lógico do usuário é transformado para evitar colisões simples
    """
    safe_name = ( # Substituições por underscores para evitar hierarquia de pasta
        path.replace("/", "_")
        .replace("\\", "_")
        .replace(":", "_")
        .replace(" ", "_")
    )
    # O nome do chunk inclui o ID do chunk formatado para garantir ordenação correta
    return f".chunks/{safe_name}/chunk_{chunk_id:06d}"
```